University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Raspberry Pi Camera Module V2

Srikanth Nanda

6. November 2025

1. Raspberry Pi Camera Module V2
2. Hardware & sensor details
3. Calibration & image corrections
4. Applications
5. Field deployment (example: PiCAM)
6. Practical tips
7. Connecting Raspberry Pi Camera Module V2 to NVIDIA Jetson Nano
8. Calibration and Testing Code for Raspberry Pi Camera Module V2

9 Applications of Raspberry Pi Camera Module V2

10 Quellen

Raspberry Pi Camera Module V2

Raspberry Pi Camera Module V2 I



Abbildung: Raspberry Pi Camera Module V2

What is the Raspberry Pi Camera Module V2? I

The **Raspberry Pi Camera Module V2** is a compact 8-megapixel camera module based on the Sony IMX219 sensor (commonly used in research and maker projects). It supports still images up to 3280x2464 and video modes up to 1080p30/720p60.

Key references: Pagnutti et al. (2017) for calibration/use in scientific imaging; Bowman et al. (2020) for flat-field / colour correction; Yang et al. (2023) for field-deployable Pi-based camera systems.

Cite examples: [PRC+17; Bow+20; Yan+23]

Hardware & sensor details

Sensor and optics I

- Sensor: Sony IMX219 (8 MP, 1.4µm pixels) used on the **Raspberry Pi Camera Module V2**.
- Interface: CSI-2 ribbon connector to Raspberry Pi single-board computers.
- Important: stock lens and firmware shading correction are tuned for the stock optics — removing the lens or adding custom optics requires calibration and flat-field correction. [Bow+20]

Calibration & image corrections

Calibration steps I

Camera intrinsics & distortion:

1. Capture chessboard / checkerboard images using the **Raspberry Pi Camera Module V2**.
2. Use OpenCV calibrateCamera() to compute intrinsic matrix K and distortion coefficients.
3. Save the calibration file and use remapping/undistort at capture time to remove lens distortion.

Flat-field and colour corrections: Bowman et al. provide methods to correct vignetting and colour crosstalk when using alternate optics (lens shading tables and colour unmixing). [Bow+20; Bow+19]

Applications

Common and research applications I

The **Raspberry Pi Camera Module V2** is widely used in:

- **Time-lapse ecology / phenology monitoring** — low-power PiCAM deployments in Arctic and remote sites. [Yan+23]
- **Low-cost photogrammetry / UAV experiments** — prototyping airborne imaging solutions and orthomosaic capture.
- **Educational vision projects and robotics** — computer vision teaching, object detection, SLAM prototyping.
- **Custom optics / scientific imaging** — with corrected flat-field & colour unmixing it becomes suitable for quantitative imaging. [PRC+17; Bow+20]

Field deployment (example: PiCAM)

PiCAM: low-power time-lapse system I

The PiCAM system uses Raspberry Pi SBC + **Raspberry Pi Camera Module V2** and low-power scheduling to capture long-term time series for plant phenology monitoring in Arctic conditions — see Yang et al. (2023) for hardware design, power management, and deployment lessons. [Yan+23]

Practical tips

Tips for better scientific images I

- Keep the stock lens if you don't need custom optics; otherwise run flat-field correction per Bowman et al. [Bow+20]
- Use RAW capture modes where possible (to avoid firmware JPEG processing) and apply your own demosaicing and corrections.
- Protect the module from moisture/temperature extremes when doing field work (PiCAM design discusses enclosures and power schedules). [Yan+23]

Connecting Raspberry Pi Camera Module V2 to NVIDIA Jetson Nano

Hardware Connection & Setup I

Required components:

- NVIDIA Jetson Nano Developer Kit (2GB or 4GB).
- Raspberry Pi Camera Module V2 (Sony IMX219).
- CSI ribbon cable (15-pin).

Connection steps:

1. Power off the Jetson Nano completely.
2. Locate the CSI camera connector (near the USB ports).
3. Gently lift the black locking tab on the connector.
4. Insert the camera's ribbon cable with the metal contacts **facing the HDMI port**.
5. Push the cable in fully, then press the locking tab down.
6. Power on the Jetson Nano.

Verification: I

Use the `ls` command to check for video devices in the `/dev/` directory.
You should see a device such as `/dev/video0`.
If you are getting an error here, the issue is likely with the GStreamer plugins or your hardware setup, not the LaTeX code itself!
Reference designs and sensor-level evaluation of the Raspberry Pi Camera Module V2 are provided by Pagnutti et al. [PRC+17] and Bowman et al. [Bow+20].

Calibration and Testing Code for Raspberry Pi Camera Module V2

Camera Calibration and Testing I

Purpose: To correct geometric distortion, flat-field variation, and colour in images from the **Raspberry Pi Camera Module V2** as recommended by [PRC+17; Bow+20].

1. Live Camera Testing

Before calibration, verify the camera feed using this simple live test script, `LiveCameraTest.py`.

```
##
# @file LiveCameraTest.py
# @brief Simple script to test live camera feed capture (e.g
    ., on Jetson/Pi).
# @author Srikanth
# @date Nov 6, 2025
##

import cv2

# Initialize video capture. 0 is usually the default built-
    in webcam.
```

Camera Calibration and Testing II

```
11 # On Jetson/Pi, this might need to be adjusted (e.g., adding
       specific API backends).
12 cap = cv2.VideoCapture(0)
13
14 # Check if the camera opened successfully
15 if not cap.isOpened():
16     print("Error: Could not open camera.")
17     exit()
18
19 # Main loop to continuously read and display frames
20 print("Starting live camera feed. Press 'ESC' to exit...")
21 while True:
22     # Read a frame from the camera
23     ret, frame = cap.read()
24
25     # If frame is not read correctly (e.g., camera
       disconnected), break the loop
26     if not ret:
27         print("Error: Failed to read frame from camera.")
28         break
```

Camera Calibration and Testing III

```
29
30      # Display the resulting frame in a window named "Live"
31      cv2.imshow("Live", frame)
32
33      # Wait for a key press for 1ms.
34      # If the key is 'ESC' (ASCII 27), break the loop
35      if cv2.waitKey(1) & 0xFF == 27:
36          break
37
38  # Release the camera and close all OpenCV windows
39  cap.release()
40  cv2.destroyAllWindows()
41  print("Camera released and windows closed.")
```

2. Capture Calibration Images

The script `CaptureCaliberationImages.py` uses the optimized GStreamer pipeline for the Jetson Nano to capture sequential checkerboard images.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Camera Calibration and Testing IV

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
@file        : CaptureCaliberationImages.py
@author      : Srikanth N
@date        : Nov 6, 2025
@version     : 1.0
@brief       : Captures calibration images from the Raspberry Pi Camera V2
               on an NVIDIA Jetson Nano using OpenCV and GStreamer.
@details     : This script utilizes the 'nvarguscamerasrc' GStreamer plugin
               for high-performance CSI camera access on the Jetson platform.
               It captures a set number of still images, saving them as 'calib_XX.jpg',
               which is suitable for camera calibration (e.g., lens distortion correction).
"""
```

Camera Calibration and Testing V

```
15 import cv2
16 import time # Added for potential use in a calibration loop
17
18 # --- Configuration Constants ---
19 # --- Doxygen Grouping Example ---
20 ## @defgroup Configuration
21 # @{
22 ## @brief The GStreamer pipeline string for the CSI camera
       on Jetson Nano.
23 #  @details This pipeline enables hardware acceleration and
       is crucial for optimal CSI camera performance.
24 #           It captures at 3264x2464 and scales down to 1280
       x720 for processing.
25 GSTREAMER_PIPELINE = (
26     "nvarguscamerasrc sensor_id=0 ! "  # Use camera sensor 0
        (CSI port 0)
27     "video/x-raw(memory:NVMM), width=3264, height=2464,
       framerate=21/1, format=NV12 ! "
28     "nvvidconv flip-method=0 ! "  # 0=none, 2=180 degrees,
       etc.
```

Camera Calibration and Testing VI

```
29      "video/x-raw, width=1280, height=720, format=BGRx ! "  # Output resolution for processing/display
30      "videoconvert ! "
31      "video/x-raw, format=BGR ! appsink"
32  )
33
34  ## @brief The total number of calibration images to capture.
35  NUM_CAPTURES = 20
36  ## @}
37
38  def capture_calibration_images():
39      """!
40      @brief Initializes the CSI camera and captures a sequence of calibration images.
41
42      @details Attempts to open the camera using the defined GStreamer pipeline.
43               If successful, it enters a loop to capture `NUM_CAPTURES` images,
```

Camera Calibration and Testing VII

```
44              saving each with a sequential filename (‘
    calib_00.jpg‘ to ‘calib_19.jpg‘).
45              It is designed to mimic the batch capture
    capability of ‘raspistill‘.
46
47      @return int : Returns 0 on success, -1 on failure to
    open the camera.
48      """
49
50      cap = cv2.VideoCapture(GSTREAMER_PIPELINE, cv2.
    CAP_GSTREAMER)
51
52      if not cap.isOpened():
53          print("Error: Could not open camera using the
    GStreamer pipeline.")
54          return -1
55
56      print("Camera opened successfully. Preparing to capture
    calibration images...")
57
```

Camera Calibration and Testing VIII

```
58      for i in range(NUM_CAPTURES):
59          # A small delay to allow a user to reposition the
        checkerboard for calibration
60          # time.sleep(1) # Uncomment if you want an automatic
         delay
61
62          ret, frame = cap.read()
63
64          if ret:
65              filename = f"calib_{i:02d}.jpg"
66              # cv2.imwrite() is used to save the image to
        disk
67              cv2.imwrite(filename, frame)
68              print(f"Captured {filename}")
69          else:
70              print("Error: Could not read frame from camera.
        Stopping capture loop.")
71              break
72
73      # Clean up resources
```

Camera Calibration and Testing IX

```
74      cap.release()
75      cv2.destroyAllWindows()
76      print("Capture complete and camera released.")
77      return 0
78
79  if __name__ == "__main__":
80      capture_calibration_images()
```

3. OpenCV Calibration Code

The distortion and flat-field calibration steps correspond to the approaches demonstrated in [Bow+20], while spectral and radiometric performance metrics follow [PRC+17]. The following script, `CameraCalibration.py`, performs the actual computation.

Camera Calibration and Testing X

```
/**
 * @file LiveCameraTest.py
 * @brief Simple script to test live camera feed capture (e.g., on Jetson/Pi).
 * @author Srikanth
 * @date Nov 6, 2025
 */

import cv2

# Initialize video capture. 0 is usually the default built-in webcam.
# On Jetson/Pi, this might need to be adjusted (e.g., adding specific API backends).
cap = cv2.VideoCapture(0)

# Check if the camera opened successfully
if not cap.isOpened():
    print("Error: Could not open camera.")
    exit()
```

Camera Calibration and Testing XI

```
18
19 # Main loop to continuously read and display frames
20 print("Starting live camera feed. Press 'ESC' to exit...")
21 while True:
22     # Read a frame from the camera
23     ret, frame = cap.read()
24
25     # If frame is not read correctly (e.g., camera
       disconnected), break the loop
26     if not ret:
27         print("Error: Failed to read frame from camera.")
28         break
29
30     # Display the resulting frame in a window named "Live"
31     cv2.imshow("Live", frame)
32
33     # Wait for a key press for 1ms.
34     # If the key is 'ESC' (ASCII 27), break the loop
35     if cv2.waitKey(1) & 0xFF == 27:
36         break
```

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Camera Calibration and Testing XII

```
37
38 # Release the camera and close all OpenCV windows
39 cap.release()
40 cv2.destroyAllWindows()
41 print("Camera released and windows closed.")
```

Applications of Raspberry Pi Camera Module V2

Example Research and Engineering Uses I

The **Raspberry Pi Camera Module V2** is a low-cost, flexible imaging device used in numerous domains:

Environmental and ecological monitoring

- The PiCAM system deploys the camera for long-term monitoring of plant phenology in Arctic ecosystems. [Yan+23]
- Combines low power scheduling, time-lapse imaging, and weatherproof housings.

Scientific imaging and calibration research

- Pagnutti et al. [PRC+17] demonstrate how to calibrate the sensor for scientific imaging, measuring radiometric linearity, spectral response, and noise.
- Bowman et al. [Bow+20] present correction models for lens shading and colour mixing that make the module suitable for quantitative microscopy and optical experiments.

Example Research and Engineering Uses II

Robotics and computer vision

- Used for object detection, visual odometry, and SLAM on Jetson Nano and Raspberry Pi.
- Integration with TensorRT or PyTorch allows real-time inference on embedded systems.

UAV and remote sensing platforms

- Mounted on drones for lightweight orthomosaic mapping and photogrammetry; see performance evaluations in [PRC+17].

Summary:

- Low cost and open software support make the **Raspberry Pi Camera Module V2** ideal for education, research, and embedded systems.
- Calibration and colour correction (Bowman et al. 2020) and low-power design (Yang et al. 2023) enable scientific-grade imaging on a budget.

Vielen Dank

für Ihre Aufmerksamkeit

Quellen

Quellen I

Nachfolgend werden die Quellen der Bilder angegeben, die für diese Präsentation in ihrer ursprünglichen Form oder modifiziert verwendet worden sind.

[Bow+19] Richard Bowman u. a. *Flat-field and colour correction for the Raspberry Pi camera module*. arXiv:1911.13295. Preprint version (instrumentation); related journal version DOI 10.5334/joh.20. 2019. DOI: 10.48550/arXiv.1911.13295.

[Bow+20] Richard W. Bowman u. a. "Flat-field and colour correction for the Raspberry Pi camera module". In: *Journal of Open Hardware* 4.1 (2020). Characterisation and practical corrections (vignetting, colour crosstalk) for Raspberry Pi Camera Module V2., S. –. DOI: 10.5334/joh.20.

Quellen II

[PRC+17] Mary Pagnutti, Robert E. Ryan, George Cazenavette u. a. "Laying the foundation to use Raspberry Pi 3 V2 camera module imagery for scientific and engineering purposes". In: *Journal of Electronic Imaging* 26.1 (2017). Evaluation and calibration of Raspberry Pi Camera Module V2 (Sony IMX219)., S. 013014. DOI: 10.1117/1.JEI.26.1.013014.

[Yan+23] Dedi Yang u. a. "PiCAM: A Raspberry Pi-based open-source, low-power camera system for monitoring plant phenology in Arctic environments". In: *Methods in Ecology and Evolution* 14.12 (2023). Field-deployable Pi-based time-lapse system using Raspberry Pi Camera Module V2 (chosen sensor) and low-power automation., S. 2974–2984. DOI: 10.1111/2041-210X.14231.